Open in app ↗

Medium

Access to everything. Now 30% off. Upgrade now

Japanese Language + Katakana + Language Learning +

Katakana Language Trainer

9 min read · 15 hours ago



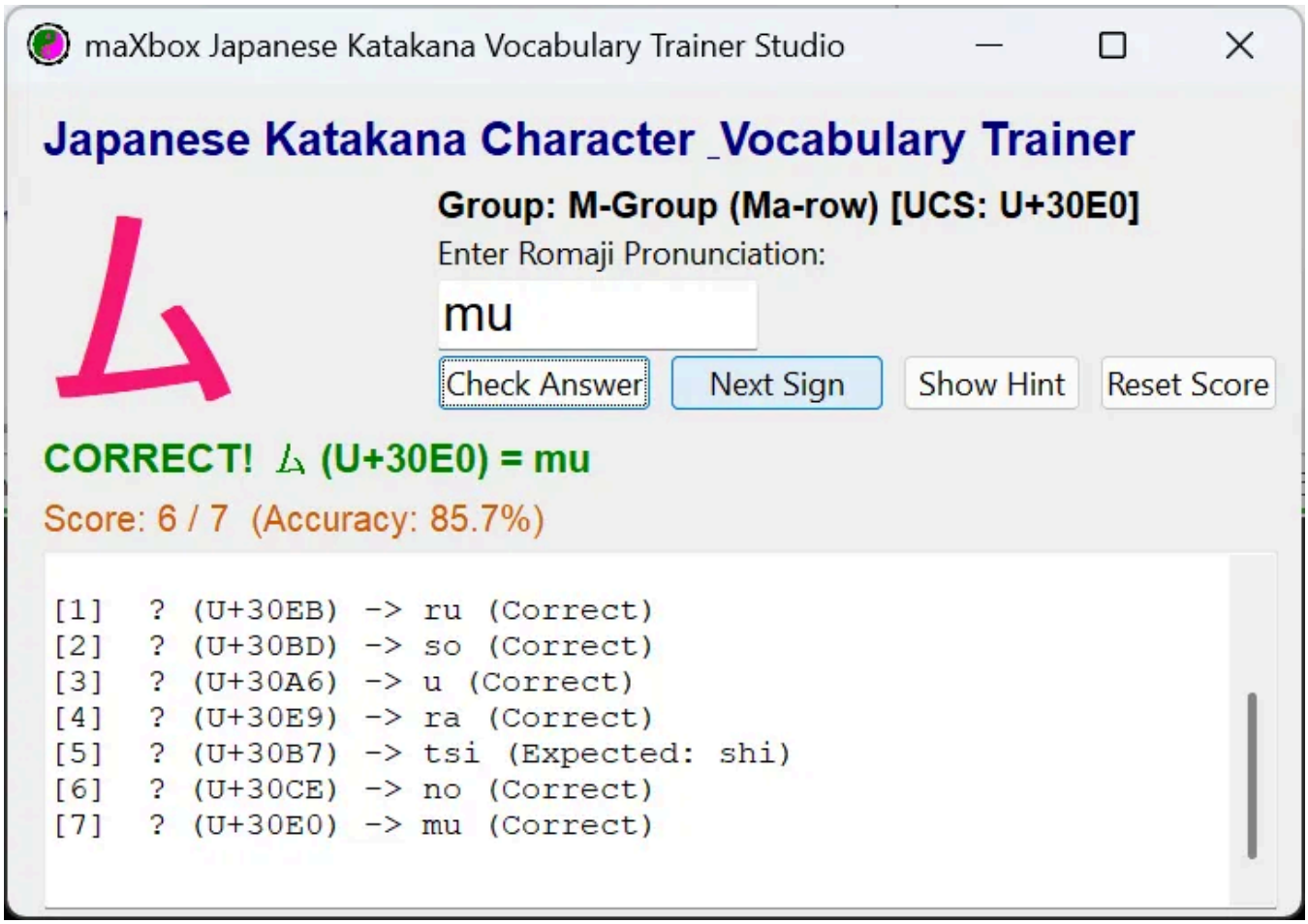
Max Kleiner

Listen Share ••• More

Katakana is a Japanese syllabary used alongside hiragana and kanji. It represents the same set of sounds as hiragana but uses distinct, more angular shapes. It originated from components of kanji during the 8th–9th centuries. Wikipedia

Structure

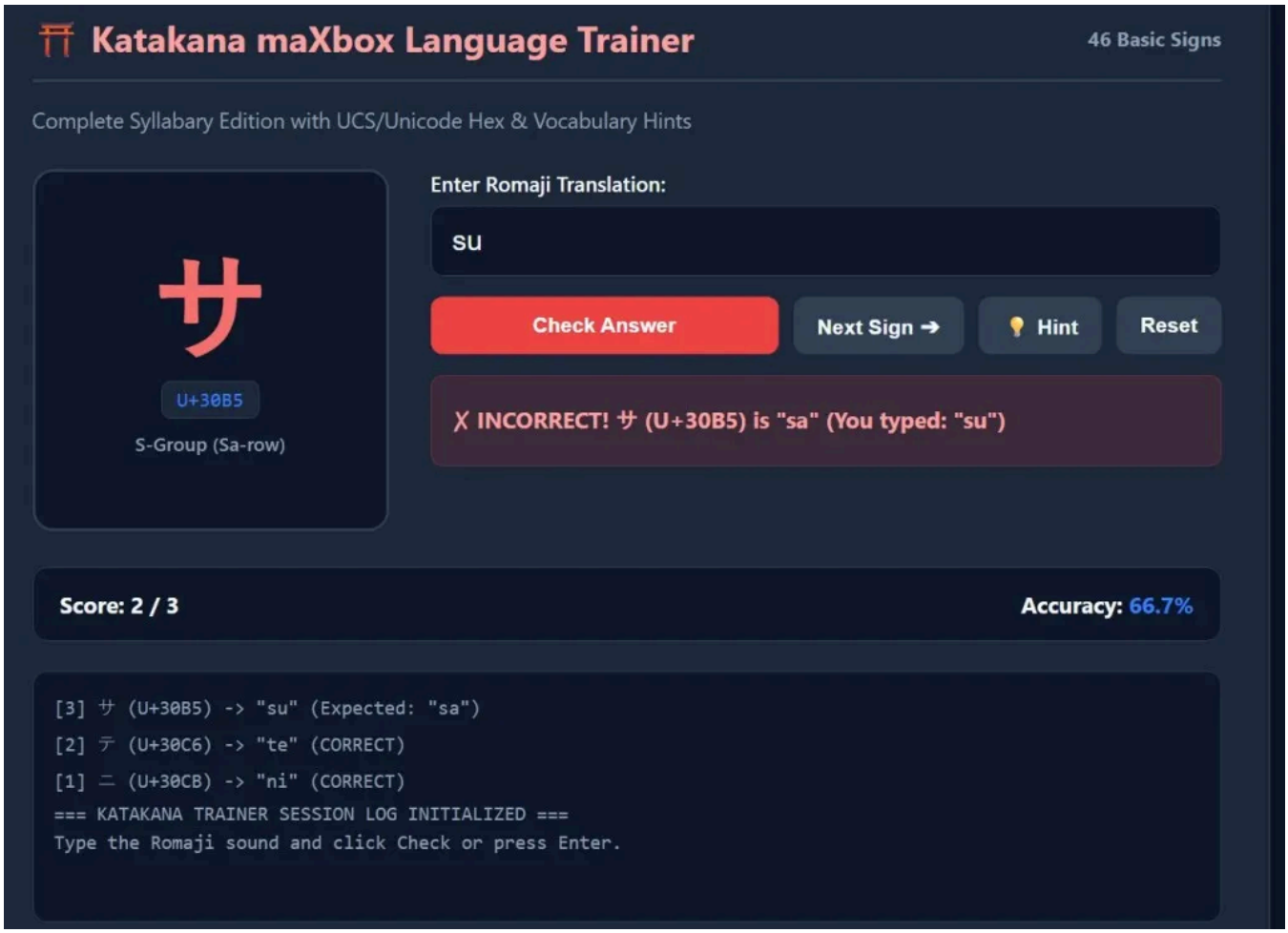
- Each character represents a mora (a rhythmic unit in Japanese).
- Includes vowels (ア, イ, ウ, エ, オ), consonant–vowel combinations (e.g., カ, サ, タ), and the nasal ン.
- Wikipedia
- Long vowels are marked with ー (chōonpu).
- Dakuten (゛) and handakuten (゜) modify consonant sounds (e.g., カ→ガ, ハ→パ).



https://sourceforge.net/projects/maxbox5/files/examples/1507_lang_trainer_katakana1.txt/download

This will be the Tutorial 205 with maXbox and JS App online.

Japanese Katakana Vocabulary & Character Trainer (46 Signs)



This script will show **Complete Syllabary Overview (katakana_language_trainer-v5.pas)**

Interactive Features: Includes randomized card drawing, real-time Romaji validation, Unicode hex code displays (**U+XXXX**), loanword vocabulary hints, accuracy tracking, and session logging.

46 Total Basic Characters: Covers Vowels (A-row), K-Group, S-Group, T-Group, N-Group, H-Group, M-Group, Y-Group, R-Group, W-Group, and Singular N.

Locale-Independent Rendering: Every character uses exact WideChar hexadecimal literals (`#$30A2` through `#$30F3`) so that maXbox renders them cleanly across all regional Windows codepages.

The most I learned from this association table with hint as the last row below. Here is the complete table of all **46 Katakana characters**, their **Unicode 16-bit hex codes**, **Romaji pronunciations**, **syllabary groups**, and corresponding **vocabulary** association **hints:**

Here is the complete table of all **46 Katakana characters**, their **Unicode 16-bit hex codes**, **Romaji pronunciations**, **syllabary groups**, and corresponding **vocabulary hints**:

Complete 46-Character Katakana Vocabulary Hint Table compressed

Katakana — Hex Code — Romaji — Syllabary Group — Vocabulary Hint & Japanese Spelling:

ア `U+30A2` **a**Vowels (A-row)**America** (アメリカ / *amerika*)**イ** `U+30A4` **i**Vowels (A-row)**India** (インド / *indo*)**ウ** `U+30A6` **u**Vowels (A-row)**Vector** (ベクトル / *beku-toru*)**エ** `U+30A8` **e**Vowels (A-row)**Energy** (エネルギ / *enerugi*)**オ** `U+30AA` **o**Vowels (A-row)**Orange** (オレンジ / *orenji*)**カ** `U+30AB` **ka**K-Group (Ka-row)**Camera** (カメラ / *kamera*)**キ** `U+30AD` **ki**K-Group (Ka-row)**King** (キング / *kingu*)**ク** `U+30AF` **ku**K-Group (Ka-row)**Class** (クラス / *kurasu*)**ケ** `U+30B1` **ke**K-Group (Ka-row)**Cake** (ケーキ / *keeki*)**コ** `U+30B3` **ko**K-Group (Ka-row)**Coffee** (コーヒー / *koohii*)**サ** `U+30B5` **sa**S-Group (Sa-row)**Salad** (サラダ / *sarada*)**シ** `U+30B7` **shi**S-Group (Sa-row)**Shirt** (シャツ / *shatsu*)**ス** `U+30B9` **su**S-Group (Sa-row)**Sports** (スポーツ / *supootsu*)**セ** `U+30BB` **se**S-Group (Sa-row)**Center** (センター / *sentaa*)**ソ** `U+30BD` **so**S-Group (Sa-row)**Sofa** (ソファー / *sofaa*)**タ** `U+30BF` **ta**T-Group (Ta-row)**Taxi** (タクシー / *takushii*)**チ** `U+30C1` **chi**T-Group (Ta-row)**Team** (チーム / *chiimu*)**ツ** `U+30C4` **tsu**T-Group (Ta-row)**Tour** (ツアー / *tsuaa*)**テ** `U+30C6` **te**T-Group (Ta-row)**Tennis** (テニス / *tenisu*)**ト** `U+30C8` **to**T-Group (Ta-row)**Tomato** (トマト / *tomato*)**ナ** `U+30CA` **na**N-Group (Na-row)**Knife** (ナイフ / *naifu*)**ニ** `U+30CB` **ni**N-Group (Na-row)**News** (ニュース / *nyuusu*)**ヌ** `U+30CC` **nu**N-Group (Na-row)**Noodle** (ヌードル / *nuudoru*)**ネ** `U+30CD` **ne**N-Group (Na-row)**Necktie** (ネクタイ / *nekutai*)**ノ** `U+30CE` **no**N-Group (Na-row)**Note** (ノート / *nooto*)**ハ** `U+30CF` **ha**H-Group (Ha-row)**Ham** (ハム / *hamu*)**ヒ** `U+30D2` **hi**H-Group (Ha-row)**Hero** (ヒロー / *hiroo*)**フ** `U+30D5` **fu**H-Group (Ha-row)**Fork** (フォーク / *fooku*)**ヘ** `U+30D8` **he**H-Group (Ha-row)**Helicopter** (ヘリコプター / *herikoputaa*)**ホ** `U+30DB` **ho**H-Group (Ha-row)**Hotel** (ホテル / *hoteru*)**マ** `U+30DE` **ma**M-Group (Ma-row)**Mask** (マスク / *masuku*)**ミ** `U+30DF` **mi**M-Group (Ma-row)**Milk** (ミルク / *miruku*)**ム** `U+30E0` **mu**M-Group (Ma-row)**Music** (ミュージック / *myuujikku*)**メ** `U+30E1` **me**M-Group (Ma-row)**Menu** (メニュー / *menyuu*)**モ** `U+30E2` **mo**M-Group (Ma-row)**Monitor** (モニター / *monitaa*)**ヤ** `U+30E4` **ya**Y-Group (Ya-row)**Yahoo** (ヤフー / *yafuu*)**ユ** `U+30E6` **yu**Y-Group (Ya-row)**User** (ユーザー / *yuuzaa*)**ヨ** `U+30E8` **yo**Y-Group (Ya-row)**Yogurt** (ヨーグルト / *yooguruto*)**ラ** `U+30E9` **ra**R-Group (Ra-row)**Radio** (ラジオ / *rajio*)**リ** `U+30EA` **ri**R-Group (Ra-row)**Ribbon** (リボン / *ribon*)**ル** `U+30EB` **ru**R-Group (Ra-row)**Rule** (ルール / *ruuru*)**レ** `U+30EC` **re**R-Group (Ra-row)**Lemon** (レモン / *remon*)**ロ** `U+30ED` **ro**R-Group (Ra-row)**Robot** (ロボッ

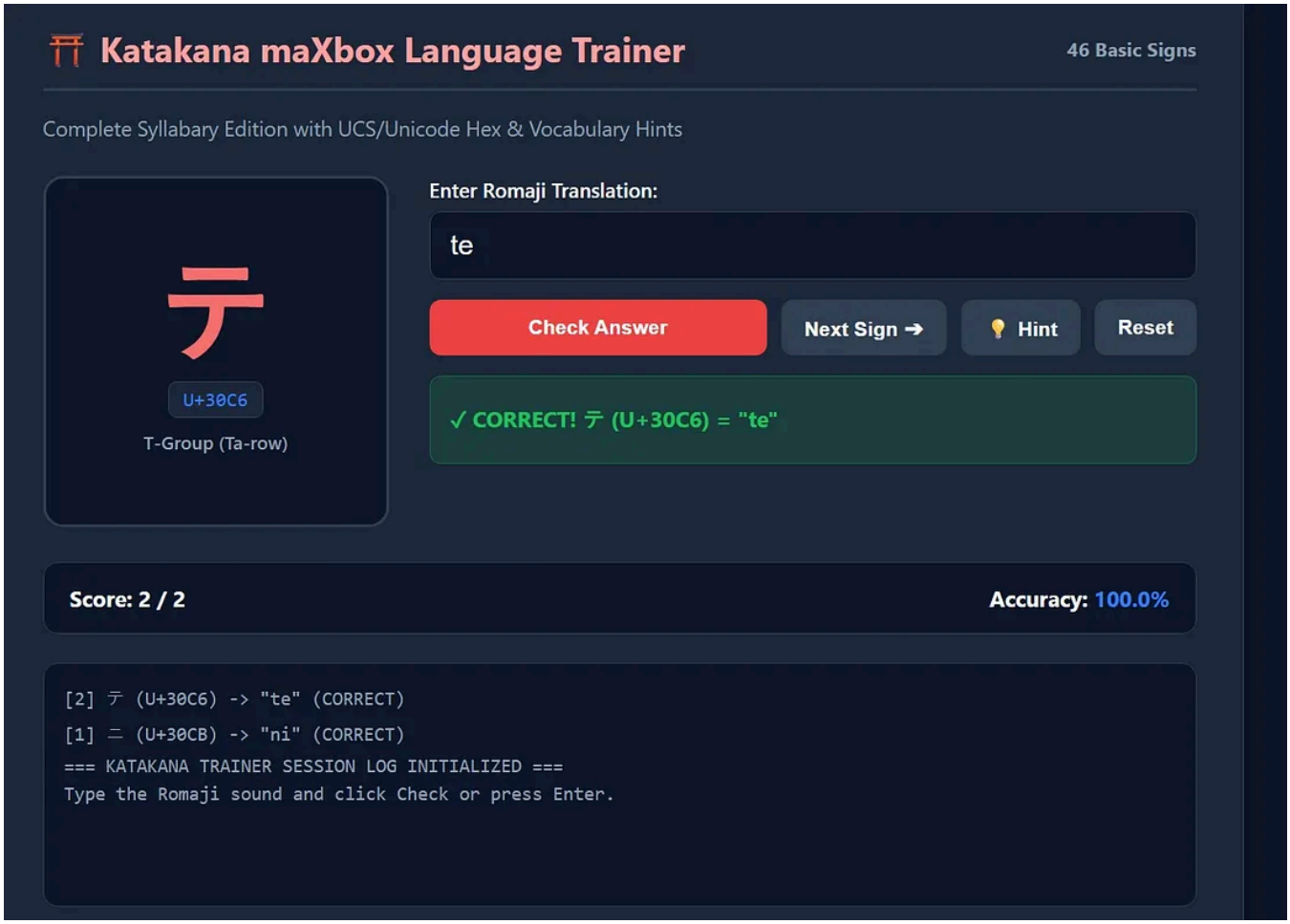
ト / *robotto*)**ワ** `U+30EF` **wa**W-Group (Wa-row)**Wine** (ワイン / *wain*)**ヲ** `U+30F2` **wo**W-Group (Wa-row)**Particle Wo** (ヲ)**ン** `U+30F3` **n**N-Singular**Pan / Bread** (パン / *pan*)

. . .

⛩️ Ref: Character Sets: Japanese Hiragana and Katakana

Key Features of this maXbox Web Application

1. **Full 46-Character Syllabary:** Incorporates all Katakana signs across 11 groups (Vowels, K, S, T, N, H, M, Y, R, W, and singular N).
2. **Unicode Hex display:** Displays the exact **U+XXXX** code point for every character.
3. **Interactive Practice Engine:** Supports keyboard (`Enter` key) navigation, immediate Romaji scoring, loanword hints, and real-time accuracy logging.



How does the script works:

The **Katakana Language Trainer** (`katakana_language_trainer-v5.pas` / `katakana_trainer_v5.html`) operates through **five core functional components:**

1. Dataset Initialization (`InitKatakanaDataset`)

- Stores all **46 basic Katakana signs** across 11 syllabary groups.
- Uses **UCS/Unicode 16-bit WideChar hex codes** (e.g., `#$30A2` for **ア**, `#$30AB` for **カ**, `#$30F3` for **ン**) to ensure exact character rendering without regional Windows ANSI code-page issues.
- Pairs each character with:

1. Its **Romaji target string** (e.g., `ka`, `shi`, `pan`).
2. Its **Syllabary Group** (e.g., `K-Group`, `N-Singular`).
3. A **Vocabulary Hint** with Japanese loanword spelling (e.g., `Pan / Bread (パン / pan)`).

2. Random Card Selection (`LoadNewCharacter`)

- Generates a pseudo-random index using `Random(46)`.
- Updates the main display label with the large 64pt Katakana character, its `U+XXXX` hex code badge, and its group title.
- Clears the input field (`EdtAnswer.Text := ''`), focuses the cursor, and sets `IsChecked := False`.

3. Answer Verification & Scoring (`CheckAnswer`)

- Sanitizes user input by trimming whitespace and converting to lowercase.
- Compares user input against the expected Romaji pronunciation:
- **Correct**: Increments `CorrectCount`, highlights feedback in **Green**, and logs `[Attempt #] Symbol (U+Hex) -> User Input (Correct)`.
- **Incorrect**: Highlights feedback in **Red** displaying expected vs. typed answers, and logs the attempt.

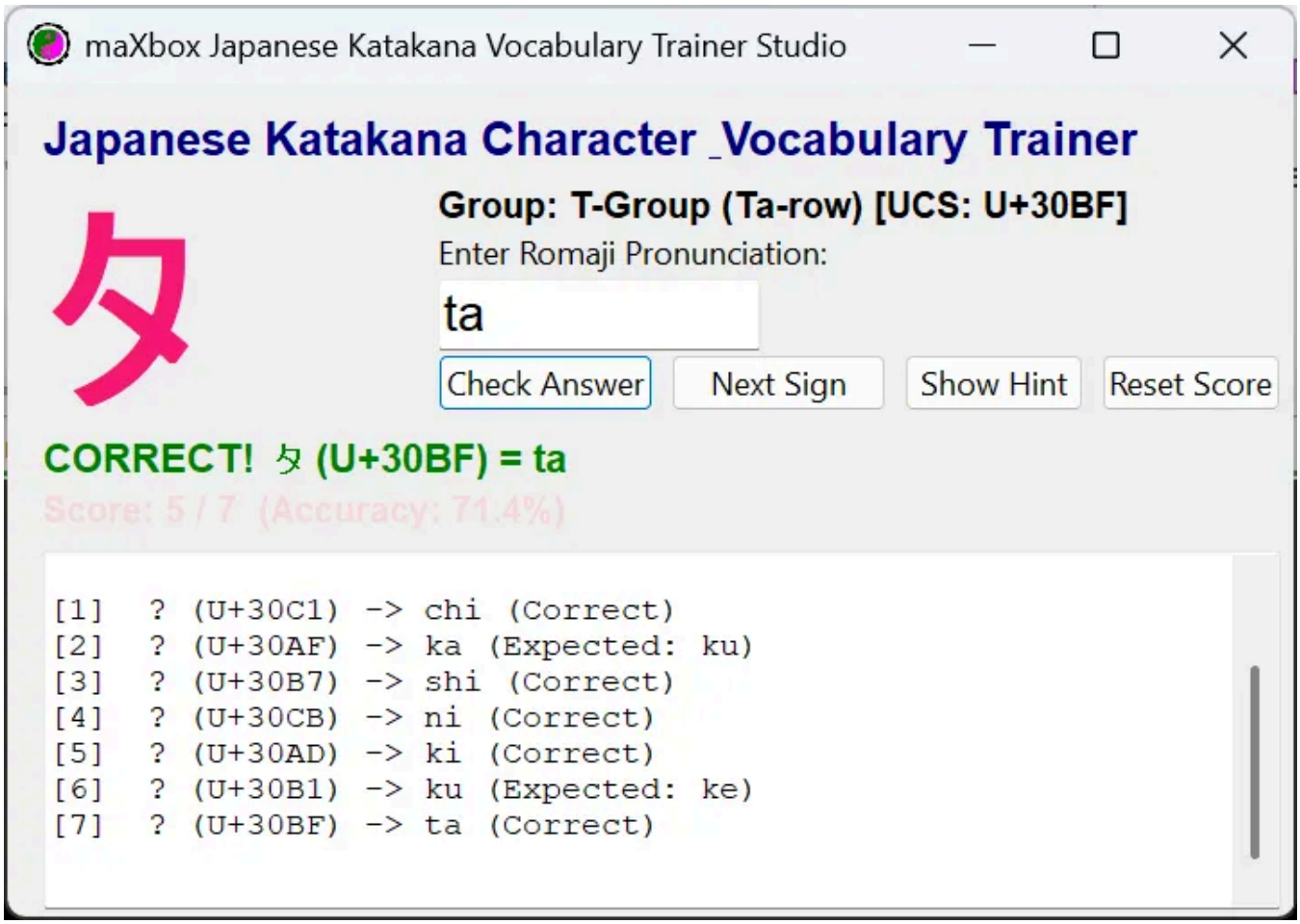
- Recalculates real-time accuracy percentage: \[\text{Accuracy %} = \frac{\text{Correct Count}}{\text{Total Attempts}} \times 100 \]

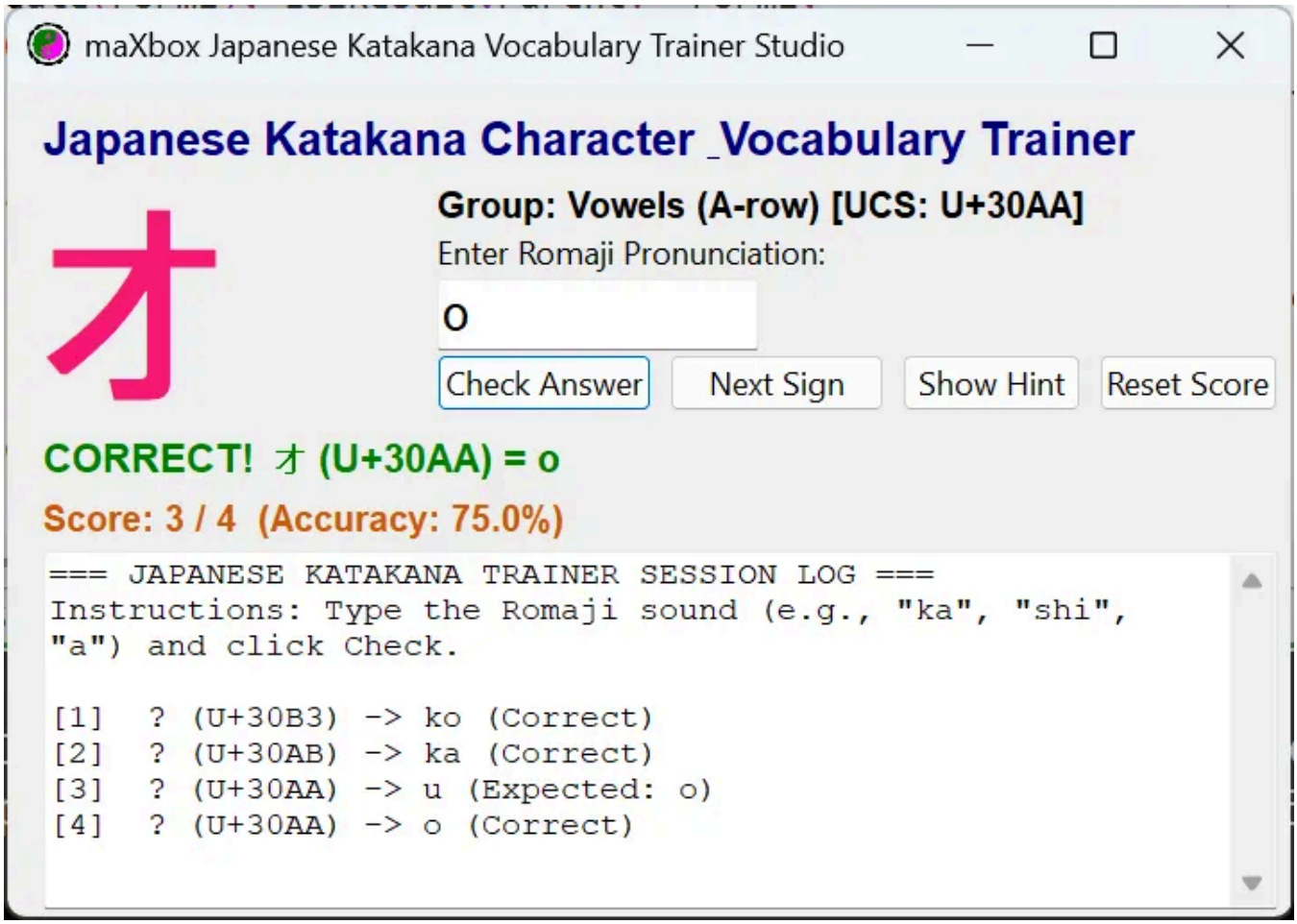
4. Hint System (`ShowHint`)

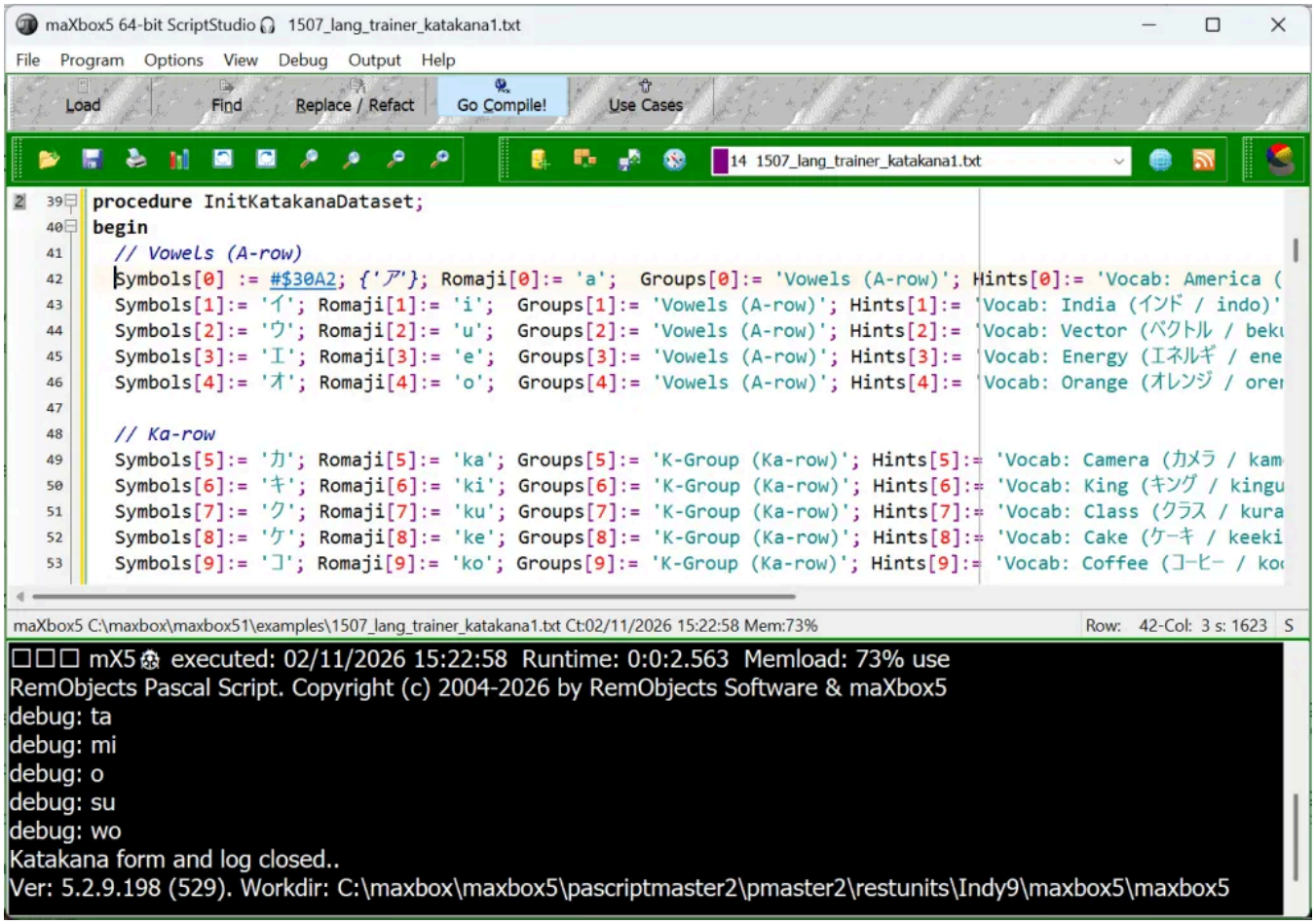
- Triggers a modal dialog displaying the character's syllabary group and its associated loanword hint without revealing the exact Romaji answer directly.

5. Event Handling & GUI Setup (`ShowGUI`)

- Instantiates window components (`TForm`, `TLabel`, `TEdit`, `TButton`, `TMemo`).
- Binds action buttons to Pascal procedure pointers (`@CheckAnswer`, `@LoadNewCharacter`, `@ShowHint`, `@ResetStats`).
- Supports rapid practice by allowing users to press **Enter** or click **Check Answer** to validate and advance to the next sign.







https://sourceforge.net/projects/maxbox5/files/examples/1507_lang_trainer_katakana1.txt/download

. . .

Sample Object Pascal Initialization Syntax (`v5`)

```
// Vowels & Ka-row
Hints := 'Vocab: America ('#$30A2#$30E1#$30EA#$30AB' / amerika)';
Hints := 'Vocab: India ('#$30A4#$30F3#$30C9' / indo)';
Hints := 'Vocab: Camera ('#$30AB#$30E1#$30E9' / kamera)';
Hints := 'Vocab: Cake ('#$30B1#$30FC#$30AD' / keeki)';
Hints := 'Vocab: Pan / Bread ('#$30D1#$30F3' / pan)';
```

```
Hints := 'Vocab: America ('#$30A2#$30E1#$30EA#$30AB' / amerika)';
```

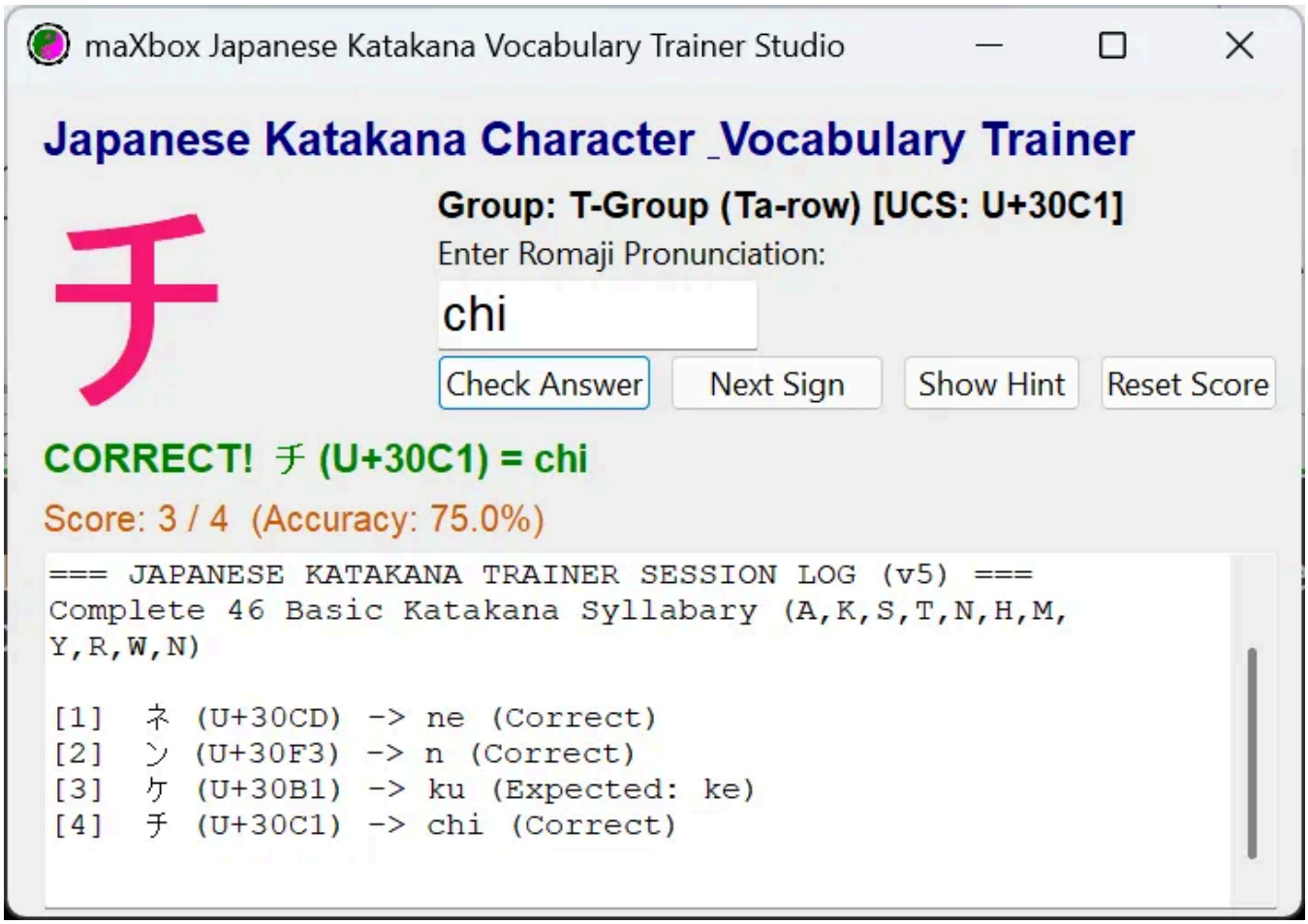
```
Hints := 'Vocab: India ('#$30A4#$30F3#$30C9' / indo)';
```

```
Hints := 'Vocab: Camera ('#$30AB#$30E1#$30E9' / kamera)';
```

```
Hints := 'Vocab: Cake ('#$30B1#$30FC#$30AD' / keeki)';
```

```
Hints := 'Vocab: Pan / Bread ('#$30D1#$30F3' / pan)';
```

🎌 Would you like to generate a practice quiz or study guide based on these Unicode Katakana vocabulary pairs?



In maXbox (and Delphi / Object Pascal), **UCS/Unicode WideChar hex literals** allow you to specify exact Unicode character code points directly in your source code without depending on system text encodings.



Here is how they work under the hood:

1. Syntax Breakdown (`#$xxxx`)

The syntax consists of three distinct compiler instructions:

- `#` : Indicates a character literal (ASCII or Unicode) rather than a number.
- `$` : Designates a **hexadecimal** (base-16) value.

- **xxxx**: The 4-digit 16-bit hexadecimal representation of the Unicode code point (e.g., `30A2` = \(12,450_{10}\)).

For example, `#$30A2` explicitly instructs the interpreter to insert the 16-bit WideChar for **U+30A2** (the Katakana character **ア**).

2. 16-Bit UCS-2 / UTF-16 Memory Representation

maXbox's PascalScript engine handles string buffers using 16-bit WideChar units (`WideString` / `UnicodeString`):

- **Basic Multilingual Plane (BMP)**: Standard characters (including Latin, Japanese Katakana/Hiragana `U+3040` – `U+30FF`, Greek, Cyrillic) fit within 16 bits (`U+0000` to `U+FFFF`). A single `#$XXXX` literal directly stores one character in memory.
- **Literal Chaining**: Multiple hex character literals can be concatenated directly alongside regular quoted text without using the `+` operator: `// Chaining hex literals for 'アメリカ' inside a string: 'Vocab: America ('#$30A2#$30E1#$30EA#$30AB' / amerika)'` Privacy Settings

3. System Code-Page & Locale Independence

The most critical reason for using `#$XXXX` hex literals is **cross-platform and locale portability**:

- **The Problem with Raw Strings**: Standard quoted text strings like `'アメリカ'` rely on the Windows system's active ANSI code page (e.g., Code Page 932 for Japanese vs. Code Page 1252 for Western European). Running raw non-ASCII string literals on non-Japanese Windows locales often corrupts characters into `?` or broken glyphs.
- **The Hex Solution**: `#$30A2` bypasses the OS code-page converter completely by writing the exact 16-bit Unicode value into the string buffer, ensuring identical rendering across all Windows regional settings.

4. Supplementary Unicode Planes & Emojis (Surrogate Pairs)

For Unicode code points above `U+FFFF` (such as Emojis in the Supplementary Multilingual Plane):

- UTF-16 splits the character into **two 16-bit surrogate units** (a high surrogate and a low surrogate).
- In maXbox scripts, emojis are written by chaining two hex literals together: `// U+1F642 (Slightly Smiling Face) = High Surrogate $D83D + Low Surrogate $DE42 Writeln(#$D83D#$DE42);`

🦋 helper tool: https://sourceforge.net/projects/maxbox5/files/examples/1507_utf8_to_pascal_hex_converter.pas/download

In **UTF-16** (the internal encoding used by maXbox `WideString` / `UnicodeString` buffers), characters are stored in **16-bit (2-byte) code units**.

While standard characters (like Latin, Greek, or Katakana `U+30A2`) fit inside a single 16-bit number (`U+0000` to `U+FFFF`), Emojis and advanced symbols live in the **Supplementary Multilingual Plane** (`U+10000` to `U+10FFFF`). Because these values exceed 16 bits, UTF-16 encodes them using a **surrogate pair**: two consecutive 16-bit `WideChar` units.

Worked Example: 🙂 Slightly Smiling Face (`U+1F642`)

Let's calculate the exact maXbox `#$XXXX#$YYYY` hex literal for `U+1F642`:

1. **Subtract `0x10000`:** $[\text{0x1F642} - \text{0x10000} = \mathbf{\text{0x0F642}}]$
2. **Extract 10-bit halves:**

- **High 10 bits:** $\text{0x0F642} \gg 10 = \mathbf{\text{0x3D}}$ (61_{10})
- **Low 10 bits:** $\text{0x0F642} \text{ AND } \text{0x3FF} = \mathbf{\text{0x242}}$ (578_{10})

1. **Calculate Surrogates:**

- **High Surrogate (W_1):** $\text{0xD800} + \text{0x3D} = \mathbf{\text{0xD83D}}$
- **Low Surrogate (W_2):** $\text{0xDC00} + \text{0x242} = \mathbf{\text{0xDE42}}$

In maXbox Object Pascal, this emoji is written by chaining the two WideChar hex units together:

```
Writeln('Hello '#$D83D#$DE42); // Displays: Hello 🙂
```

```
function SurrogateToCodePoint(const HighSurr, LowSurr: Word): Cardinal;
begin
  Result := ((HighSurr - $D800) shl 10) + (LowSurr - $DC00) + $10000;
end;
```

If you have a surrogate pair (\(W_1, W_2\)) in maXbox and want to reconstruct the original 32-bit Unicode code point, like above.

Conclusion

We've built a robust, fully localized **Katakana Syllabary & Vocabulary Trainer** (`katakana_language_trainer-v6.pas`) for maXbox. Here is a structural review of how the script evolved, its core architecture, and key technical highlights:

1. Complete 46-Character Syllabary Coverage

The dataset covers the entire basic Japanese Katakana syllabary across **11 distinct phonetic groups**:

- **Vowels (A-row):** ア, イ, ウ, エ, オ
- **K-Group (Ka-row):** カ, キ, ク, ケ, コ
- **S-Group (Sa-row):** サ, シ, ス, セ, ソ
- **T-Group (Ta-row):** タ, チ, ツ, テ, ト
- **N-Group (Na-row):** ナ, ニ, ヌ, ネ, ノ
- **H-Group (Ha-row):** ハ, ヒ, フ, ヘ, ホ
- **M-Group (Ma-row):** マ, ミ, ム, メ, モ
- **Y-Group (Ya-row):** ヤ, ユ, ヨ
- **R-Group (Ra-row):** ラ, リ, ル, レ, ロ

- **W-Group & Singular N:** ワ, ヲ, ン

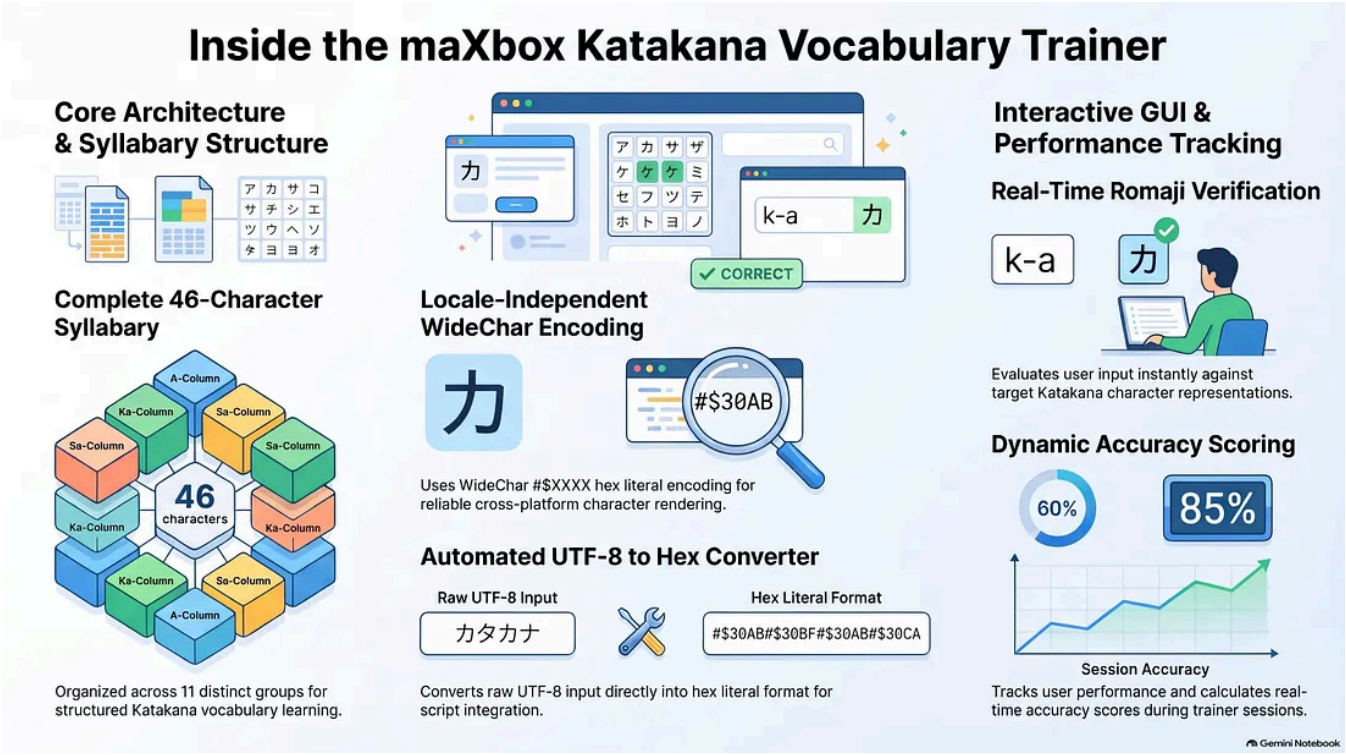
2. 100% Locale-Independent WideChar Hex Encoding

To prevent ANSI code-page rendering corruption (`?` or broken glyphs) across different Windows regional settings:

- **Character Symbols:** Each character is declared via WideChar hex literals (e.g., `#$30A2` for **ア** through `#$30F3` for **ン**).
- **Vocabulary Hints:** All Japanese loanword spellings are constructed using WideChar hex concatenations: `// America (アメリカ) Hints := 'Vocab: America ('#$30A2#$30E1#$30EA#$30AB' / amerika)'; // Bread/Pan (パン) Hints := 'Vocab: Pan / Bread ('#$30D1#$30F3' / pan)';`

3. GUI Engine & User Experience

- **Dynamic Card Renderer:** Picks a random card (`Random(46)`), updating the large **64pt Katakana display**, group category, and the **U+XXXX** hex tag.
- **Validation & Instant Feedback:** Evaluates user Romaji input against the expected string, rendering results in **Green (Correct)** or **Red (Incorrect)** with correct answer reveal.
- **Real-time Accuracy Calculation:** $$ \text{Accuracy %} = \frac{\text{Correct Answers}}{\text{Total Attempts}} \times 100 $$
- **Session Memo Logger:** Logs every attempt with attempt index, Katakana symbol, Unicode hex tag, user input, and result status for review.



Supervised Learning

Japanese Language + Katakana + Language Learning +



Follow

Written by Max Kleiner

54 followers · 3 following

Max Kleiner's professional environment is in the areas of OOP, UML and coding - among other things as a trainer, developer and consultant.